

PyTorch 卷积网络配置与训练 (一)



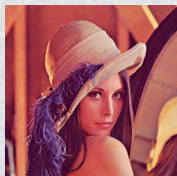
人工智能与Python程序设计 教研组

卷积网络的训练流程



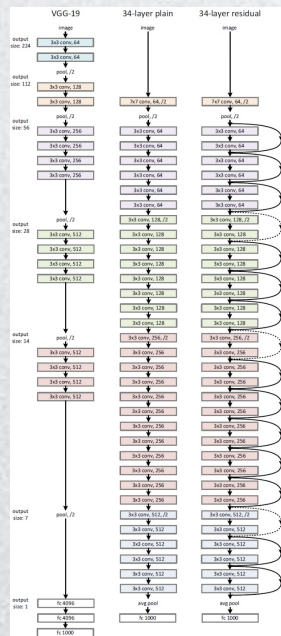
卷积网络的训练流程

数据准备



It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness...

网络定义



损失函数

Classification

Log Loss

Focal Loss

KL Divergence/
Relative
Entropy

Exponential
Loss

Hinge Loss

Regression

Mean Square
Error/
Quadratic Loss

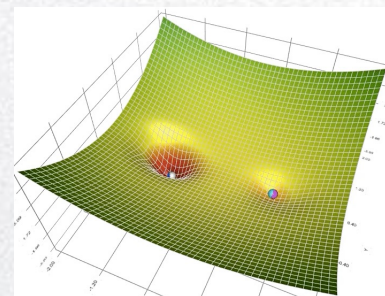
Mean Absolute
Error

Huber Loss/
Smooth Mean
Absolute Error

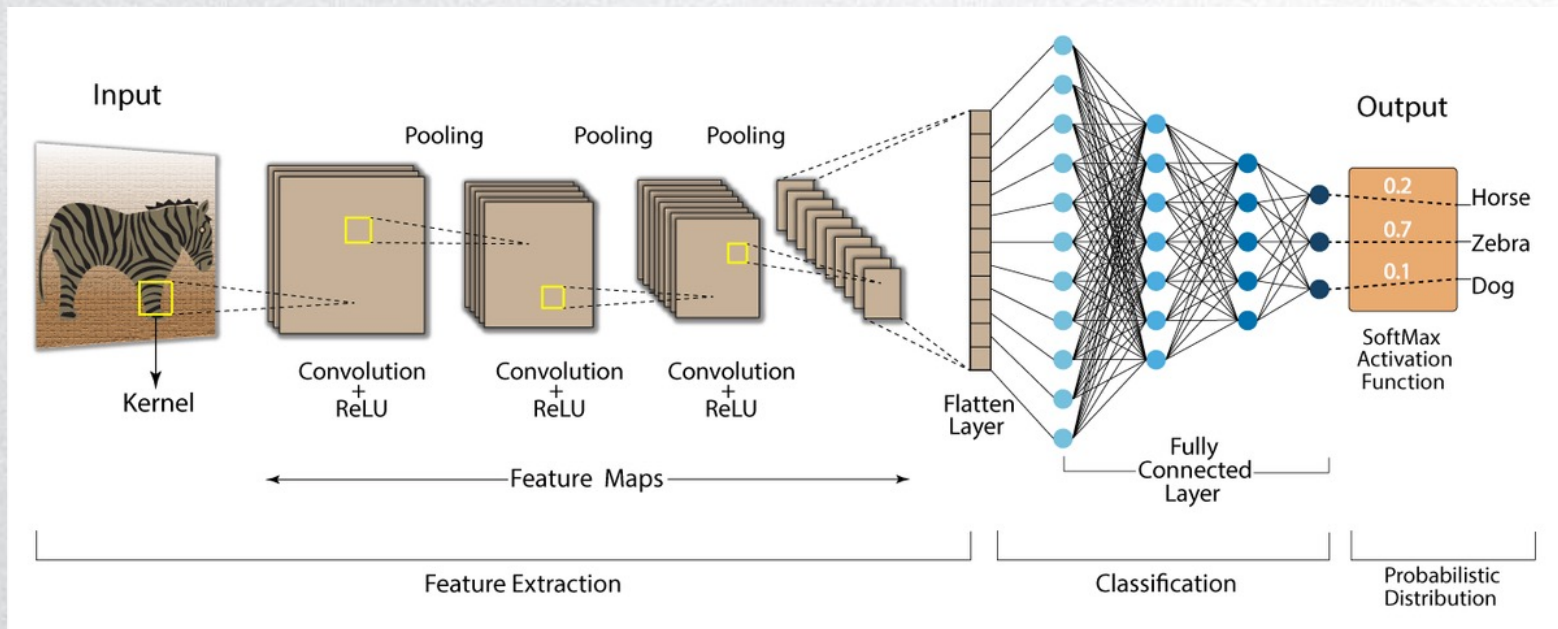
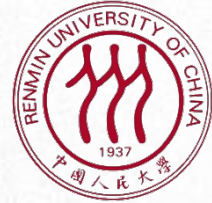
Log cosh Loss

Quantile Loss

优化方法



网络搭建与模型优化





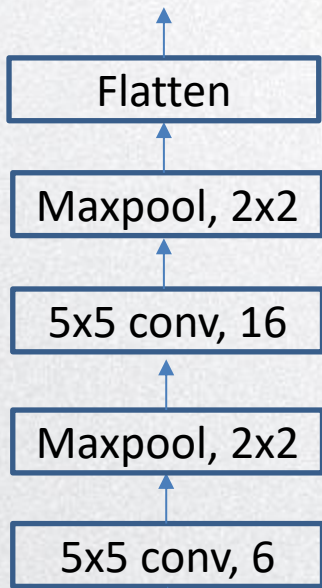
网络搭建与模型优化

```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

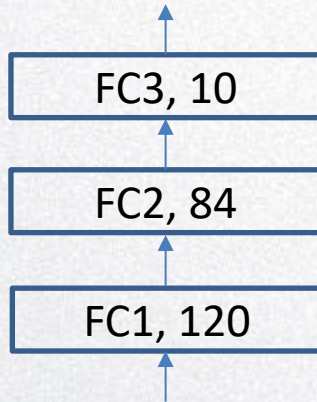
```
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```



输入 x
(N,3,32,32)

类别
预测





网络搭建与模型优化

在构造函数中，
实例化不同的
layer组件，并赋
给类成员变量

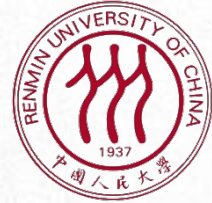
```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```

在前馈函数中，利用实例化的
组件对网络进行搭建，并对输
入Tensor进行操作，并返回
Tensor类型的输出结果



网络搭建与模型优化

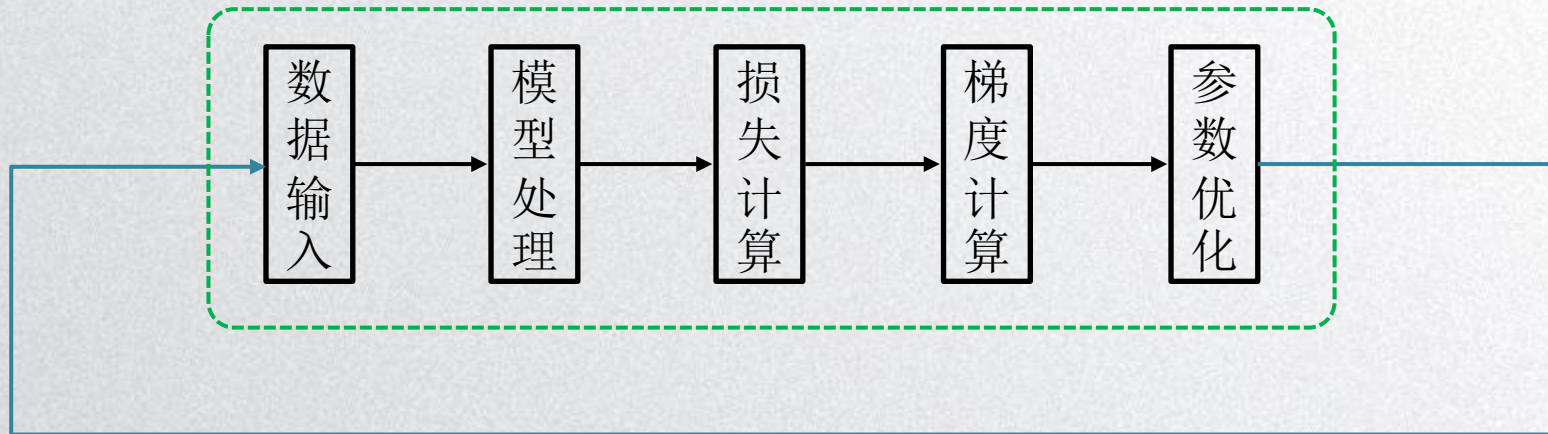
- 模型训练

网络定义

损失函数

优化方法

单批量数据下的一次模型训练



完整数据下的迭代模型训练



网络搭建与模型优化

- 模型训练

全部数据
迭代次数

```
for epoch in range(2): # loop over the dataset multiple times
```

```
    running_loss = 0.0
```

```
    for i, data in enumerate(trainloader, 0):
```

```
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data
```

获取当前批次数据

```
        # zero the parameter gradients
        optimizer.zero_grad()
```

清空模型参数的梯度

```
        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

模型预测与参数优化

```
    # print statistics
```

```
    running_loss += loss.item()
```

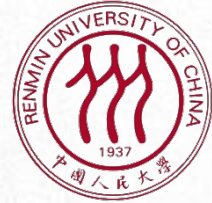
```
    if i % 2000 == 1999: # print every 2000 mini-batches
```

```
        print('[%d, %5d] loss: %.3f' %
```

```
              (epoch + 1, i + 1, running_loss / 2000))
```

```
        running_loss = 0.0
```

```
print('Finished Training')
```

PyTorch 卷积网络配置与训练 (二)



人工智能与Python程序设计 教研组



提纲



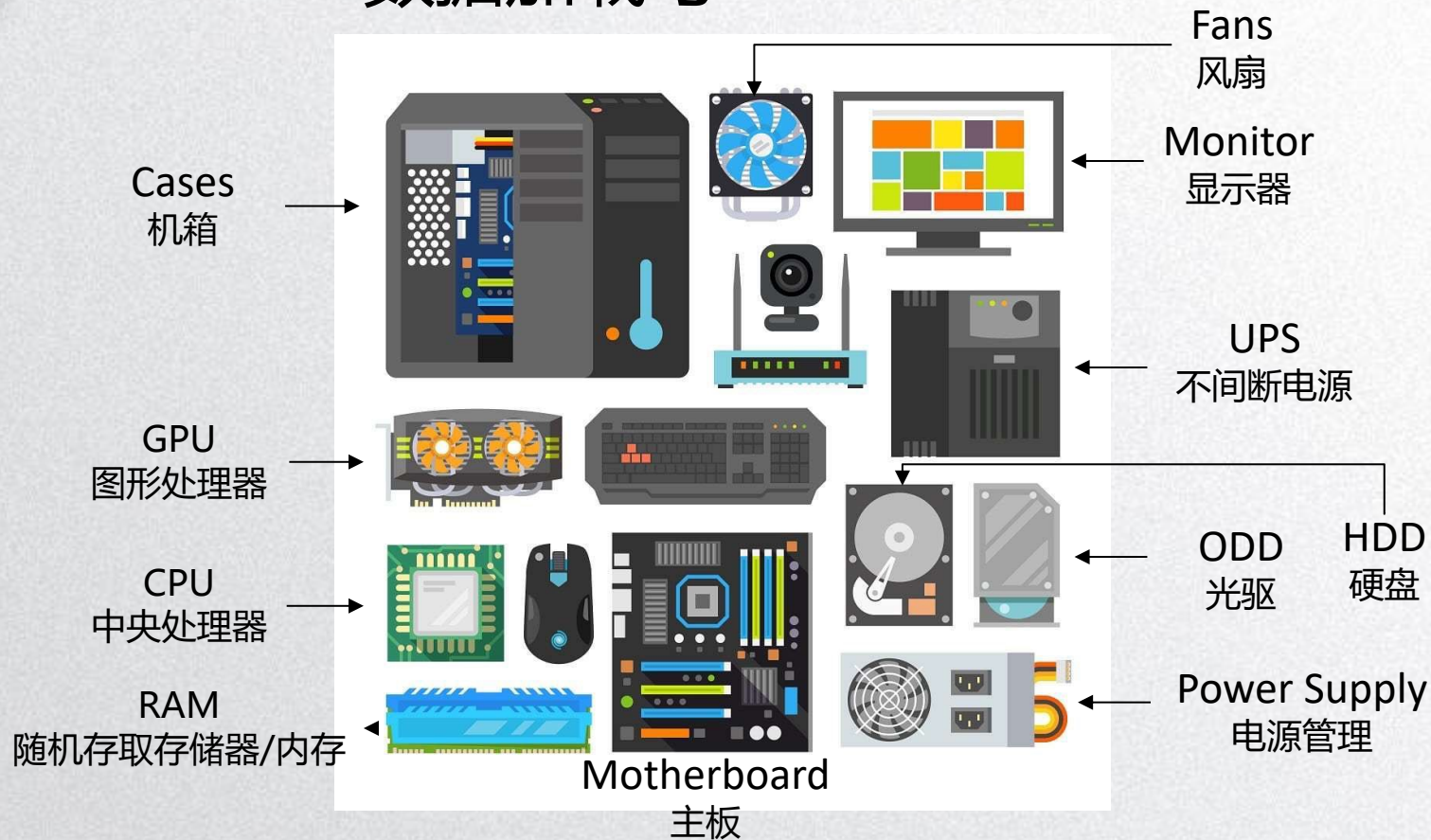
人工智能与
Python程序设计

- 1. 数据加载与DataLoader
- 2. VGG网络的搭建
- 3. 模型训练与测试
- 4. 图像识别大作业

数据加载与DataLoader



数据加载与DataLoader



数据加载与DataLoader

数据存储

数据读取

数据预处理

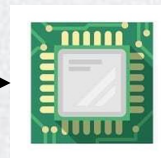
数据计算



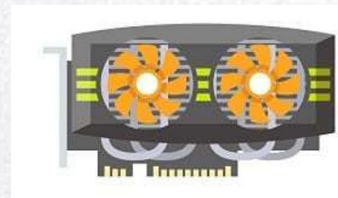
硬盘



内存



CPU



GPU



数据加载与DataLoader

单次数据加载示例：

```
# Load entire dataset
X, y = torch.load('some_training_set_with_labels.pt')
batch_size = 8

# Train model
for epoch in range(max_epochs):
    for i in range(n_batches):
        # Local batches and labels
        local_X, local_y = X[i*batch_size:(i+1)*batch_size,], y[i*batch_size:(i+1)*batch_size,]

        # Your model
        [...]
```



数据加载与DataLoader

单次数据加载示例：

```
# Load entire dataset
X, y = torch.load('some_training_set_with_labels.pt')
batch_size = 8

# Train model
for epoch in range(max_epochs):
    for i in range(n_batches):
        # Local batches and labels
        local_X, local_y = X[i*batch_size:(i+1)*batch_size:], y[i*batch_size:(i+1)*batch_size:]

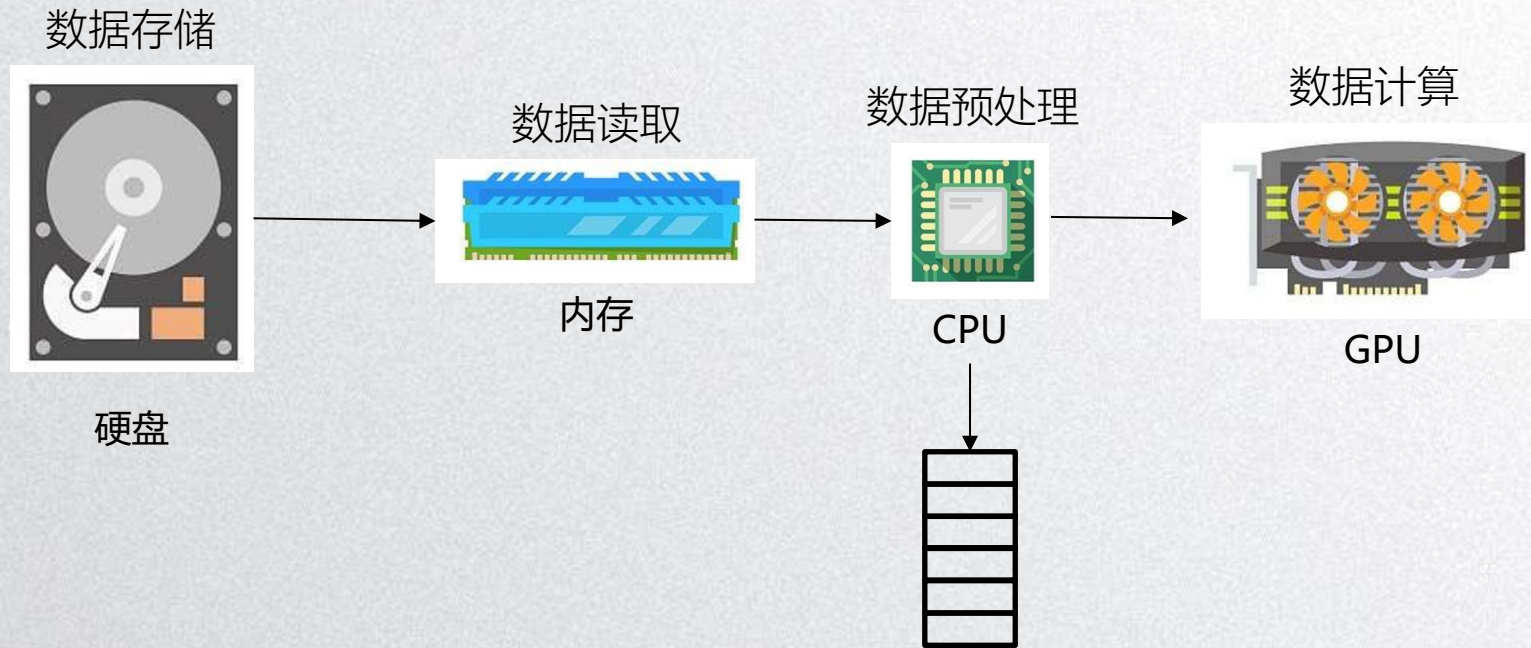
        # Your model
        [...]
```

数据传输过载与空闲

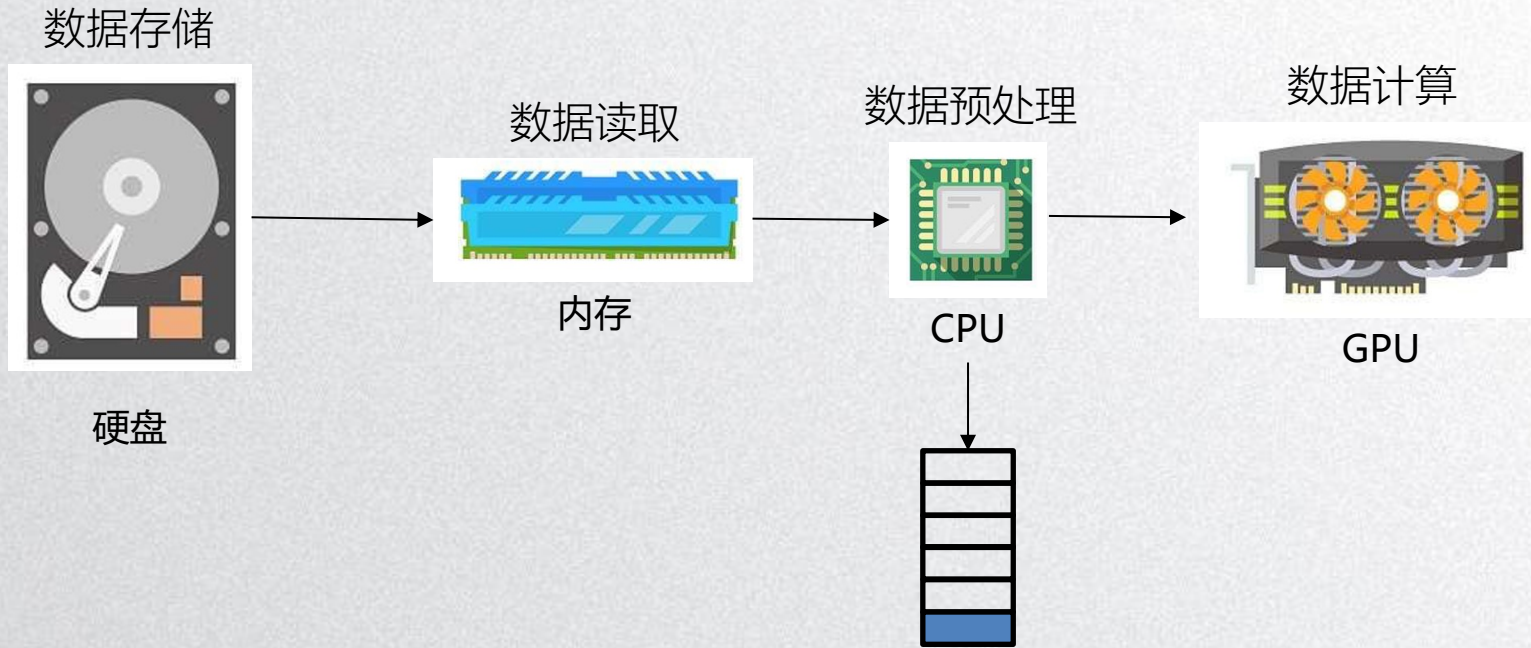
内存需求量大

GPU利用率不高

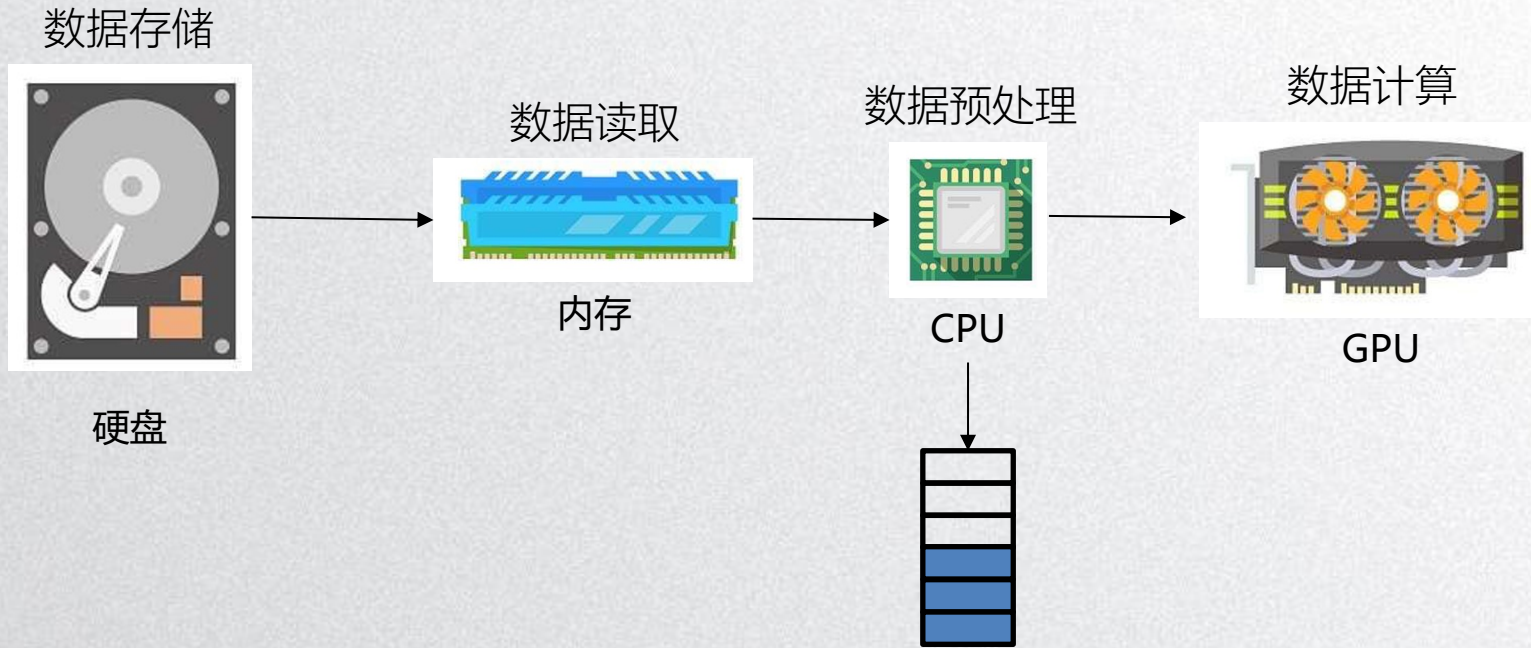
数据加载与DataLoader



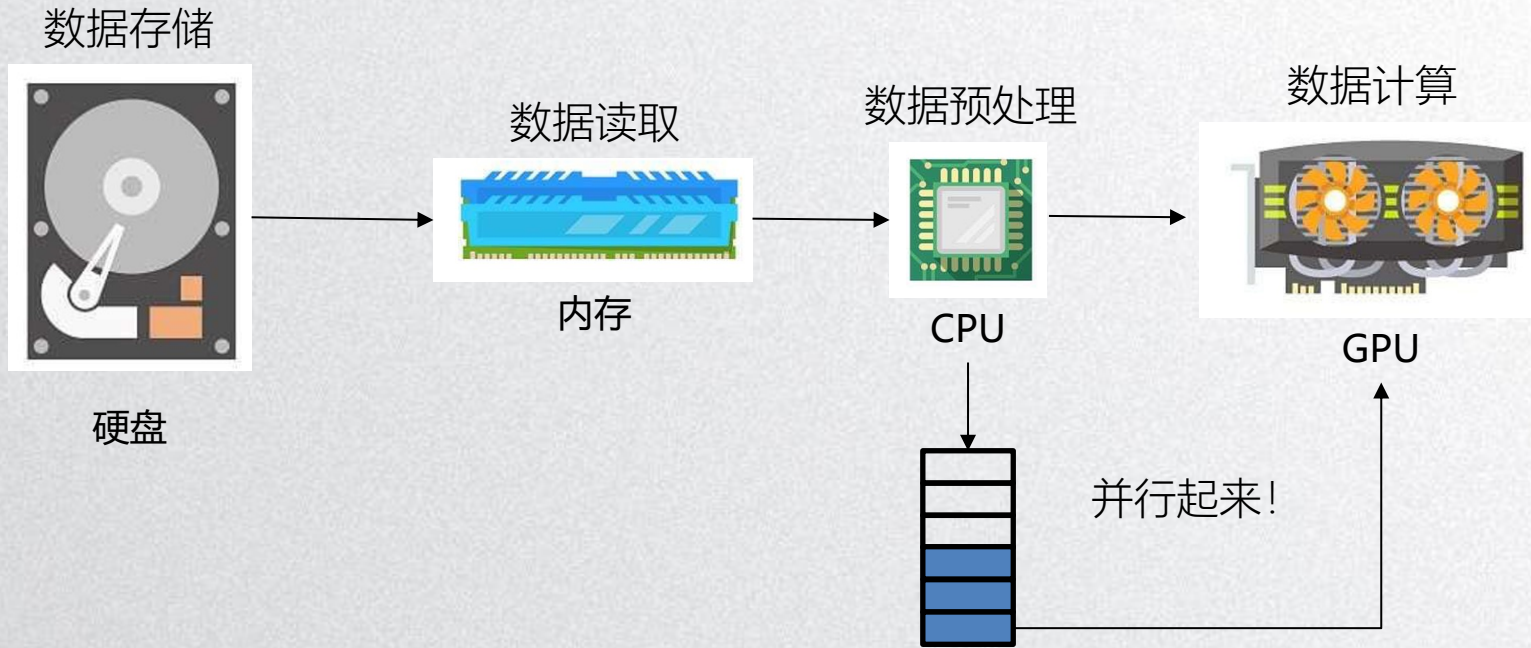
数据加载与DataLoader



数据加载与DataLoader



数据加载与DataLoader





数据加载与DataLoader



```
import torch

class Dataset(torch.utils.data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDS, labels):
        'Initialization'
        self.labels = labels
        self.list_IDS = list_IDS

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDS)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDS[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```


数据加载与DataLoader

数据个数声明

```
import torch

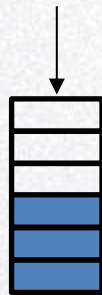
class Dataset(torch.utils.data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDs, labels):
        'Initialization'
        self.labels = labels
        self.list_IDs = list_IDs

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDs)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDs[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```



根据index获取对应数据样本，并返回到数据队列



数据加载与DataLoader

- PyTorch中的DataLoader类

```
CLASS torch.utils.data.DataLoader(dataset: torch.utils.data.dataset.Dataset[T_co],  
    batch_size: Optional[int] = 1, shuffle: bool = False, sampler:  
    Optional[torch.utils.data.sampler.Sampler[int]] = None, batch_sampler:  
    Optional[torch.utils.data.sampler.Sampler[Sequence[int]]] = None, num_workers:  
    int = 0, collate_fn: Callable[List[T], Any] = None, pin_memory: bool = False,  
    drop_last: bool = False, timeout: float = 0, worker_init_fn: Callable[int, None]  
    = None, multiprocessing_context=None, generator=None, *, prefetch_factor: int =  
    2, persistent_workers: bool = False)
```

[SOURCE]



数据加载与DataLoader

- PyTorch中的DataLoader类

```
CLASS torch.utils.data.DataLoader(dataset: torch.utils.data.dataset.Dataset[T_co],  
batch_size: Optional[int] = 1, shuffle: bool = False, sampler:  
Optional[torch.utils.data.sampler.Sampler[int]] = None, batch_sampler:  
Optional[torch.utils.data.sampler.Sampler[Sequence[int]]] = None, num_workers:  
int = 0, collate_fn: Callable[List[T], Any] = None, pin_memory: bool = False,  
drop_last: bool = False, timeout: float = 0, worker_init_fn: Callable[int, None]  
= None, multiprocessing_context=None, generator=None, *, prefetch_factor: int =  
2, persistent_workers: bool = False)
```

[SOURCE]

- 数据加载器，集成了数据集和采样器，并在数据集上提供单进程或多进程迭代器。
 - dataset: 加载数据的数据集
 - batch_size: 每个batch加载多少个样本，默认：1
 - shuffle: 设置为True时，在每个epoch重新打乱数据，默认：False
 - sampler: 定义从数据集中提取样本的策略，默认：None
 - num_worker: 用多少子进程加载数据。0表示数据将在主进程中加载，默认：0



数据加载与DataLoader

- PyTorch中的Dataset类

CLASS `torch.utils.data.Dataset`

[SOURCE]

An abstract class representing a `Dataset`.

All datasets that represent a map from keys to data samples should subclass it. All subclasses should overwrite `__getitem__()`, supporting fetching a data sample for a given key. Subclasses could also optionally overwrite `__len__()`, which is expected to return the size of the dataset by many `Sampler` implementations and the default options of `DataLoader`.

- 数据集类：表示Dataset的抽象类
 - 数据集加载前，都应该进行子类化
 - 子类化过程中，应重写类方法`__len__`和`__getitem__`
 - `__len__`：指定数据集的大小
 - `__getitem__`：实现数据样例的索引

数据加载与DataLoader

- 利用PyTorch对数据进行加载

```
import torch

class Dataset(torch.utils.data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDS, labels):
        'Initialization'
        self.labels = labels
        self.list_IDS = list_IDS

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDS)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDS[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```



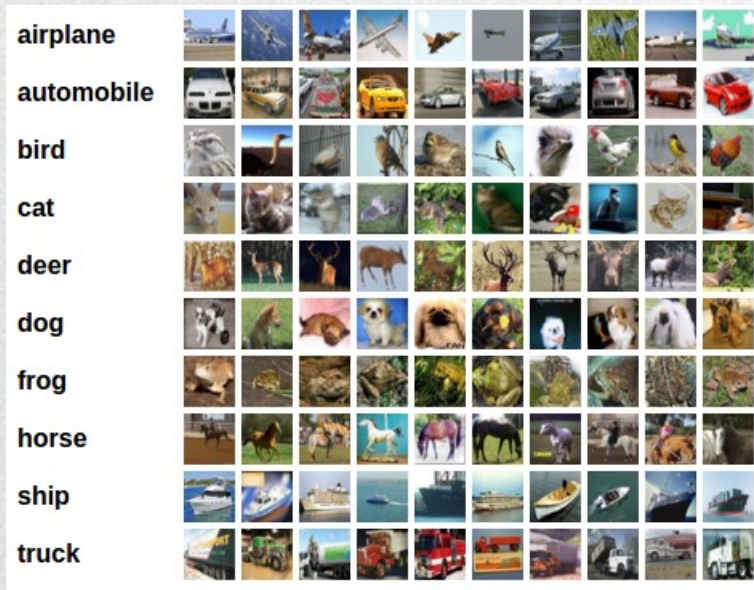
```
import torch
from my_classes import Dataset

# Datasets
data_samples = # IDs
labels = # Labels

# Generators
training_set = Dataset(data_samples['train'], labels)
training_generator = torch.utils.data.DataLoader(training_set, **params)
```

CIFAR-10数据介绍与加载

- CIFAR-10数据介绍
 - 10个类别，每个类别6000张图像，共计60000张。
 - 图像分辨率为32x32, 均为RGB三通道图像。





CIFAR-10数据介绍与加载

- 基于torchvision库对CIFAR-10数据加载

```
import torch
import torchvision
```

```
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True)
```

```
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                           shuffle=True, num_workers=2)
```

```
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True)
```

```
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                          shuffle=False, num_workers=2)
```

数据集类
实例化

DataLoader
实例化



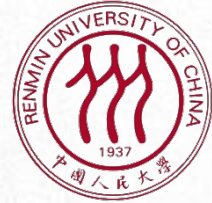
CIFAR-10数据介绍与加载

- 基于torchvision库对CIFAR-10数据加载

根据index获取对应数据样本，并返回到数据队列

```
class CIFAR10(VisionDataset):  
    .  
    .  
    .  
    def __getitem__(self, index: int) -> Tuple[Any, Any]:  
        """  
        Args:  
            index (int): Index  
  
        Returns:  
            tuple: (image, target) where target is index of the target class.  
        """  
        img, target = self.data[index], self.targets[index]  
  
        # doing this so that it is consistent with all other datasets  
        # to return a PIL Image  
        img = Image.fromarray(img)  
  
        if self.transform is not None:  
            img = self.transform(img)  
  
        if self.target_transform is not None:  
            target = self.target_transform(target)  
  
        return img, target  
  
    def __len__(self) -> int:  
        return len(self.data)
```

数据个数声明



提纲



人工智能与
Python程序设计

- 1. 数据加载与DataLoader
- 2. VGG网络的搭建
- 3. 模型训练与测试
- 4. 图像识别大作业

VGG网络的搭建



Published as a conference paper at ICLR 2015

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan^{*} & Andrew Zisserman^{*}
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen,az}@robots.ox.ac.uk

ABSTRACT

In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16-19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the localisation and classification tracks respectively. We also show that our representations generalise well to other datasets, where they achieve state-of-the-art results. We have made our two best-performing ConvNet models publicly available to facilitate further research on the use of deep visual representations in computer vision.

1 INTRODUCTION

Convolutional networks (ConvNets) have recently enjoyed a great success in large-scale image and video recognition (Krizhevsky et al., 2012; Zeiler & Fergus, 2013; Sermanet et al., 2014; Simonyan & Zisserman, 2014) which has become possible due to the large public image repositories, such as ImageNet (Deng et al., 2009), and high-performance computing systems, such as GPUs or large-scale distributed clusters (Dean et al., 2012). In particular, an important role in the advance of deep visual recognition architectures has been played by the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) (Russakovsky et al., 2014), which has served as a testbed for a few generations of large-scale image classification systems, from high-dimensional shallow feature encodings (Perotini et al., 2010) (the winner of ILSVRC-2011) to deep ConvNets (Krizhevsky et al., 2012) (the winner of ILSVRC-2012).

With ConvNets becoming more of a commodity in the computer vision field, a number of attempts have been made to improve the original architecture of Krizhevsky et al. (2012) in a bid to achieve better accuracy. For instance, the best-performing submissions to the ILSVRC-2013 (Zeiler & Fergus, 2013; Sermanet et al., 2014) utilised smaller receptive window size and smaller stride of the first convolutional layer. Another line of improvements dealt with training and testing the networks densely over the whole image and over multiple scales (Sermanet et al., 2014; Howard, 2014). In this paper, we address another important aspect of ConvNet architecture design – its depth. To this end, we fit other parameters of the architecture, and steadily increase the depth of the network by adding more convolutional layers, which is feasible due to the use of very small (3×3) convolution filters in all layers.

As a result, we come up with significantly more accurate ConvNet architectures, which not only achieve the state-of-the-art accuracy on ILSVRC classification and localisation tasks, but are also applicable to other image recognition datasets, where they achieve excellent performance even when used as a part of a relatively simple pipelines (e.g. deep features classified by a linear SVM without fine-tuning). We have released our two best-performing models¹ to facilitate further research.

The rest of the paper is organised as follows. In Sect. 2, we describe our ConvNet configurations. The details of the image classification training and evaluation are then presented in Sect. 3, and the

¹Current affiliation: Google DeepMind ²Current affiliation: University of Oxford and Google DeepMind
http://www.robots.ox.ac.uk/~vgg/research/very_deep/

Published as a conference paper at ICLR 2015

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as “conv(receptive field size)-(number of channels)”. The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A+LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
Input (224 × 224 RGB image)					
conv3-64	conv3-64	conv3-64	conv3-64	conv3-64	conv3-64
	LRN	conv3-64	conv3-64	conv3-64	conv3-64
maxpool					
conv3-128	conv3-128	conv3-128	conv3-128	conv3-128	conv3-128
		conv3-128	conv3-128	conv3-128	conv3-128
maxpool					
conv3-256	conv3-256	conv3-256	conv3-256	conv3-256	conv3-256
		conv3-256	conv3-256	conv3-256	conv3-256
maxpool					
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
		conv3-512	conv3-512	conv3-512	conv3-512
maxpool					
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
		conv3-512	conv3-512	conv3-512	conv3-512
maxpool					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A	A+LRN	B	C	D	E
Number of parameters	133	133	134	138	144	

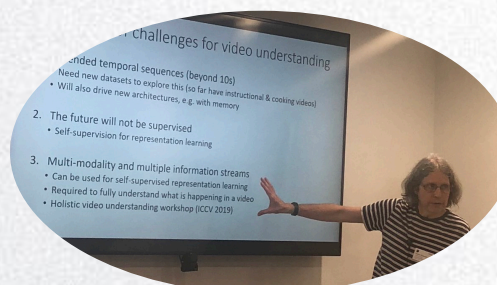
such layers have a 7×7 effective receptive field. So what have we gained by doing, for instance, a stack of three 3×3 conv. layers instead of a single 7×7 layer? First, we incorporate three non-linear rectification layers instead of a single one, which makes the decision function more discriminative. Second, we decrease the number of parameters: assuming that both the input and the output of a three-layer 3×3 convolution stack has C channels, the stack is parameterised by $3(3C^2) = 27C^2$ weights; at the same time, a single 7×7 conv. layer would require $7^2 C^2 = 49C^2$ parameters, i.e. 81% more. This can be seen as imposing a regularisation on the 7×7 conv. filters, forcing them to have a decomposition through the 3×3 filters (with non-linearity injected in between).

The incorporation of 1×1 conv. layers (configuration C, Table 1) is a way to increase the non-linearity of the decision function without affecting the receptive fields of the conv. layers. Even though in our case the 1×1 convolution is essentially a linear projection onto the space of the same dimensionality (the number of input and output channels is the same), an additional non-linearity is introduced by the rectification function. It should be noted that 1×1 conv. layers have recently been utilised in the “Network in Network” architecture of Lin et al. (2014).

Small-size convolution filters have been previously used by Ciresan et al. (2011), but their nets are significantly less deep than ours, and they did not evaluate on the large-scale ILSVRC dataset. Goodfellow et al. (2014) applied deep ConvNets (11 weight layers) to the task of street number recognition, and showed that the increased depth led to better performance. GoLeNet (Szegedy et al., 2014), a top-performing entry of the ILSVRC-2014 classification task, was developed independently of our work, but is similar in that it is based on very deep ConvNets

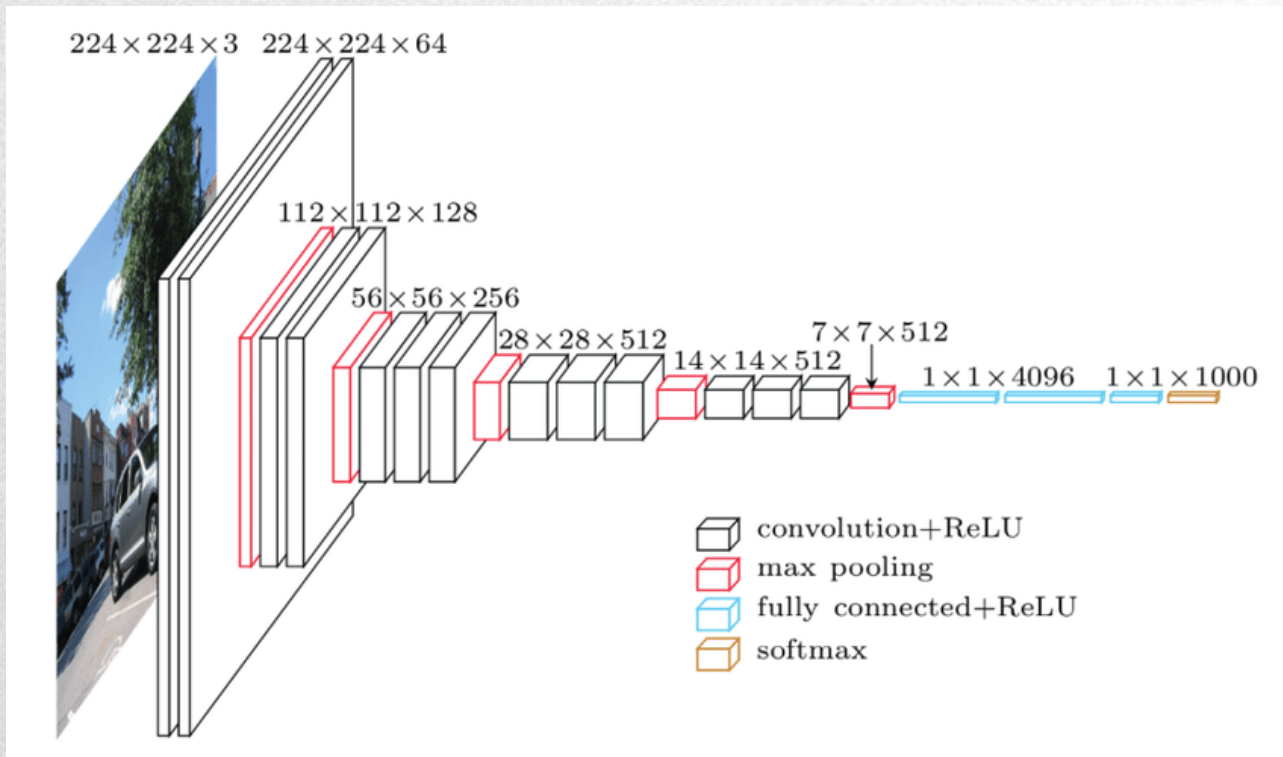


Karen Simonyan



Andrew Zisserman

VGG网络的搭建



VGG网络的搭建

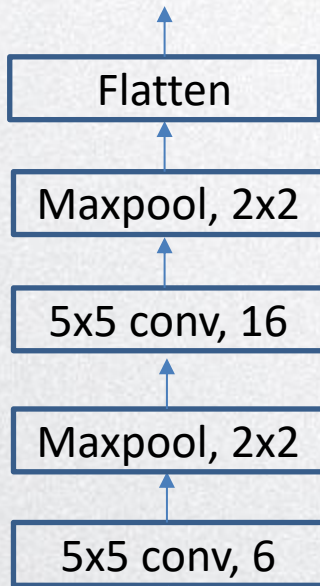
- Recall: 用表格简写表示网络架构

```
import torch.nn as nn
import torch.nn.functional as F

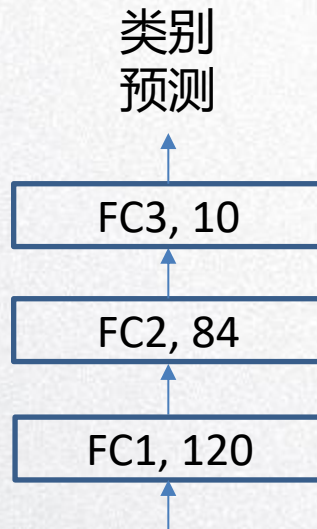
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```



输入 x
(N,3,32,32)



VGG网络的搭建

• VGG网络架构

- 基本组件：卷积层，池化层，全连接层
- 采取模块堆叠形式
- 衍生不同模型，兼顾浅层与深层网络

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

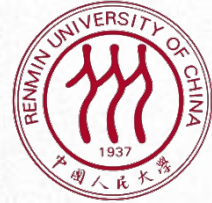
VGG网络的搭建

• VGG网络架构

- 以A架构为例
- A架构有11个带有可学习权重的层所构成，即8个卷积层和3个全连接层
- 所有卷积层均为3x3卷积

思考：C架构中的1x1卷积目的是什么？

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					



VGG网络的搭建

- VGG代码的模块设计
 - 网络架构抽象

```
cfgs: Dict[str, List[Union[str, int]]] = {  
    'A': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'B': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'D': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 'M'],  
    'E': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 'M'],  
}
```

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64	conv3-64	conv3-64	conv3-64
maxpool					
conv3-128	conv3-128	conv3-128	conv3-128	conv3-128	conv3-128
maxpool					
conv3-256	conv3-256	conv3-256	conv3-256	conv3-256	conv3-256
conv3-256	conv3-256	conv3-256	conv3-256 conv1-256	conv3-256 conv3-256	conv3-256
maxpool					
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512 conv1-512	conv3-512 conv3-512	conv3-512
maxpool					
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512 conv1-512	conv3-512 conv3-512	conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG网络的搭建

- VGG代码的模块设计
 - 假设构建A架构模型

```
cfgs: Dict[str, List[Union[str, int]]] = {  
    'A': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'B': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'D': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M'],  
    'E': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 'M'],  
}
```

```
def vgg11(pretrained: bool = False, progress: bool = True, **kwargs: Any) -> VGG:  
    """VGG 11-layer model (configuration "A") from  
    `Very Deep Convolutional Networks For Large-Scale Image Recognition` <https://arxiv.org/pdf/1409.1556.pdf>`..  
    Args:  
        pretrained (bool): If True, returns a model pre-trained on ImageNet  
        progress (bool): If True, displays a progress bar of the download to stderr  
    """  
    return _vgg('vgg11', 'A', False, pretrained, progress, **kwargs)
```

VGG网络的搭建

- VGG代码的模块设计

- 假设构建A架构模型
- 通过调用_vgg()函数, 传递相应架构编号 “A”

```
cfgs: Dict[str, List[Union[str, int]]] = {  
    'A': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'B': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'D': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 'M'],  
    'E': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 'M'],  
}
```

```
def vgg11(pretrained: bool = False, progress: bool = True, **kwargs: Any) -> VGG:  
    r"""VGG 11-layer model (configuration "A") from  
    `Very Deep Convolutional Networks For Large-Scale Image Recognition` <https://arxiv.org/pdf/1409.1556.pdf>`..  
    Args:  
        pretrained (bool): If True, returns a model pre-trained on ImageNet  
        progress (bool): If True, displays a progress bar of the download to stderr  
    """  
    return _vgg('vgg11', 'A', False, pretrained, progress, **kwargs)
```

VGG网络的搭建

- VGG代码的模块设计

- 假设构建A架构模型
- 通过调用_vgg()函数, 传递相应架构编号 “A”
- 在_vgg()函数中, 根据字典cfgs[" A"], 获取具体网络架构, 即 [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']

```
cfgs: Dict[str, List[Union[str, int]]] = {  
    'A': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'B': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'D': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M'],  
    'E': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 'M'],  
}
```

```
def _vgg(arch: str, cfg: str, batch_norm: bool, pretrained: bool, progress: bool, **kwargs: Any) -> VGG:  
    if pretrained:  
        kwargs['init_weights'] = False  
    model = VGG(make_layers(cfgs[cfg], batch_norm=batch_norm), **kwargs)  
    if pretrained:  
        state_dict = load_state_dict_from_url(model_urls[arch],  
                                              progress=progress)  
        model.load_state_dict(state_dict)  
    return model
```


VGG网络的搭建

- VGG代码的模块设计

- 在make_layers函数中，解析网络架构 [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'], 搭建网络

```
def make_layers(cfg: List[Union[str, int]], batch_norm: bool = False) -> nn.Sequential:
    layers: List[nn.Module] = []
    in_channels = 3
    for v in cfg:
        if v == 'M':
            layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
        else:
            v = cast(int, v)
            conv2d = nn.Conv2d(in_channels, v, kernel_size=3, padding=1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(v), nn.ReLU(inplace=True)]
            else:
                layers += [conv2d, nn.ReLU(inplace=True)]
            in_channels = v
    return nn.Sequential(*layers)
```



VGG网络的搭建

- VGG代码的模块设计

- `nn.Sequential()` 将网络部分层组合成一个网络module, 以同网络其他部分进行拼接

```
def make_layers(cfg: List[Union[str, int]], batch_norm: bool = False) -> nn.Sequential:
    layers: List[nn.Module] = []
    in_channels = 3
    for v in cfg:
        if v == 'M':
            layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
        else:
            v = cast(int, v)
            conv2d = nn.Conv2d(in_channels, v, kernel_size=3, padding=1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(v), nn.ReLU(inplace=True)]
            else:
                layers += [conv2d, nn.ReLU(inplace=True)]
            in_channels = v
    return nn.Sequential(*layers)
```

VGG网络的搭建

• VGG代码的模块设计

```
class VGG(nn.Module):

    def __init__(
        self,
        features: nn.Module,
        num_classes: int = 1000,
        init_weights: bool = True
    ) -> None:
        super(VGG, self).__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d((7, 7))
        self.classifier = nn.Sequential(
            nn.Linear(512 * 7 * 7, 4096),
            nn.ReLU(True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(True),
            nn.Dropout(),
            nn.Linear(4096, num_classes),
        )
        if init_weights:
            self._initialize_weights()
```

```
def make_layers(cfg: List[Union[str, int]], batch_norm: bool = False) -> nn.Sequential:
    layers: List[nn.Module] = []
    in_channels = 3
    for v in cfg:
        if v == 'M':
            layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
        else:
            v = cast(int, v)
            conv2d = nn.Conv2d(in_channels, v, kernel_size=3, padding=1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(v), nn.ReLU(inplace=True)]
            else:
                layers += [conv2d, nn.ReLU(inplace=True)]
            in_channels = v
    return nn.Sequential(*layers)
```

卷积部分

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG网络的搭建

• VGG代码的模块设计

池化与全连接部分

```
class VGG(nn.Module):

    def __init__(
        self,
        features: nn.Module,
        num_classes: int = 1000,
        init_weights: bool = True
    ) -> None:
        super(VGG, self).__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d((7, 7))
        self.classifier = nn.Sequential(
            nn.Linear(512 * 7 * 7, 4096),
            nn.ReLU(True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(True),
            nn.Dropout(),
            nn.Linear(4096, num_classes),
        )
        if init_weights:
            self._initialize_weights()
```

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					



VGG网络的搭建

- VGG代码的模块设计

```
class VGG(nn.Module):  
  
    def __init__(  
        self,  
        features: nn.Module,  
        num_classes: int = 1000,  
        init_weights: bool = True  
    ) -> None:  
        super(VGG, self).__init__()  
        self.features = features  
        self.avgpool = nn.AdaptiveAvgPool2d((7, 7))  
        self.classifier = nn.Sequential(  
            nn.Linear(512 * 7 * 7, 4096),  
            nn.ReLU(True),  
            nn.Dropout(),  
            nn.Linear(4096, 4096),  
            nn.ReLU(True),  
            nn.Dropout(),  
            nn.Linear(4096, num_classes),  
        )  
        if init_weights:  
            self._initialize_weights()
```

CLASS torch.nn.AdaptiveAvgPool2d(output_size)

[SOURCE]

Applies a 2D adaptive average pooling over an input signal composed of several input planes.

The output is of size H x W, for any input size. The number of output features is equal to the number of input planes.

将输入feature map池化为指定大小

根据被池化后的大小，构建全连接层



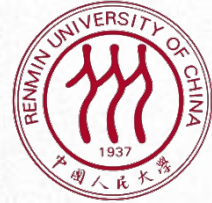
VGG网络的搭建

- VGG代码的模块设计

```
class VGG(nn.Module):  
  
    def __init__(  
        self,  
        features: nn.Module,  
        num_classes: int = 1000,  
        init_weights: bool = True  
    ) -> None:  
        super(VGG, self).__init__()  
        self.features = features  
        self.avgpool = nn.AdaptiveAvgPool2d((7, 7))  
        self.classifier = nn.Sequential(  
            nn.Linear(512 * 7 * 7, 4096),  
            nn.ReLU(True),  
            nn.Dropout(),  
            nn.Linear(4096, 4096),  
            nn.ReLU(True),  
            nn.Dropout(),  
            nn.Linear(4096, num_classes),  
        )  
        if init_weights:  
            self._initialize_weights()
```

VGG网络前馈计算

```
def forward(self, x: torch.Tensor) -> torch.Tensor:  
    x = self.features(x)  
    x = self.avgpool(x)  
    x = torch.flatten(x, 1)  
    x = self.classifier(x)  
    return x
```



VGG网络的搭建

- VGG代码的模块设计

```
class VGG(nn.Module):... VGG模型类，整合卷积module和全连接module
```

```
def make_layers(cfg: List[Union[str, int]], batch_norm: bool = False) -> nn.Sequential:... 根据网络配置，搭建卷积module
```

```
cfgs: Dict[str, List[Union[str, int]]] = {  
    'A': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'B': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],  
    'D': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M'],  
    'E': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 'M'],  
}  
各种网络配置构成的字典
```

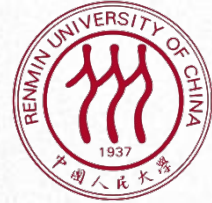
```
def _vgg(arch: str, cfg: str, batch_norm: bool, pretrained: bool, progress: bool, **kwargs: Any) -> VGG:...
```

根据给定参数，实例化VGG网络

```
def vgg11(pretrained: bool = False, progress: bool = True, **kwargs: Any) -> VGG:  
    r"""VGG 11-layer model (configuration "A") from  
    "Very Deep Convolutional Networks For Large-Scale Image Recognition" <https://arxiv.org/pdf/1409.1556.pdf>`...  
    Args:  
        pretrained (bool): If True, returns a model pre-trained on ImageNet  
        progress (bool): If True, displays a progress bar of the download to stderr  
    """  
    return _vgg('vgg11', 'A', False, pretrained, progress, **kwargs)
```

创建网络vgg11

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					



提纲



人工智能与
Python程序设计

- 1. 数据加载与DataLoader
- 2. VGG网络的搭建
- 3. 模型训练与测试
- 4. 图像识别大作业



模型训练与测试



```
import torch
import torch.nn as nn
from torchvision.models.vgg import vgg11
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
```

```
# 卷积神经网络实例化
net = vgg11(num_classes=10, pretrained=False, progress=True).to(device)
print(net)
```

从torchvision导入模型创建函数

模型创建



模型训练与测试



```
# 卷积神经网络训练
for epoch in range(epochs):
    running_loss = 0.0

    for i, data in enumerate(trainLoader, 0):
        inputs, labels = data
        # 将数据迁移到device中, 如device为GPU, 则将数据从CPU迁移到GPU; 如device为CPU, 则将数据从CPU迁移到CPU (即不作移动)
        inputs, labels = inputs.to(device), labels.to(device)

        # 清空参数的梯度值
        optimizer.zero_grad()

        # 前馈+反馈+优化
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # 统计训练损失
        running_loss += loss.item()

    if i % 2000 == 1999: # 每2000 mini-batches, 打印一次
        print('[%d, %5d] loss: %.3f' %
              (epoch + 1, i + 1, running_loss / 2000))
        running_loss = 0.0
```

模型训练



模型训练与测试

- 模型测试
 - 获取测试集的分类准确率

停止参数的
梯度计算

```
# 卷积神经网络测试
correct = 0
total = 0
with torch.no_grad(): # 测试阶段，停止梯度计算
    for data in testLoader:
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device) #

        # 前馈获得网络预测结果
        outputs = net(inputs)
        # 在预测结果中，获得预测值最大化的类别id
        _, predicted = torch.max(outputs.data, 1)

        # 统计样本个数
        total += labels.size(0)
        # 统计正确预测样本个数
        correct += (predicted == labels).sum().item()
```

获取预测类别

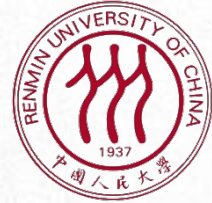


模型测试与评估

- 模型测试
 - 获取各类别的准确率

```
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))
with torch.no_grad():
    for data in testLoader:
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        # 前馈获得网络预测结果
        outputs = net(inputs)
        # 在预测结果中, 获得预测值最大化的类别id
        _, predicted = torch.max(outputs.data, 1)

        # 对于batch内每一个样本, 获取其预测是否正确
        c = (predicted == labels).squeeze()
        # 对于batch内样本, 统计每个类别的预测正确率
        for i in range(Batch_size):
            label = labels[i]
            class_correct[label] += c[i].item()
            class_total[label] += 1
```

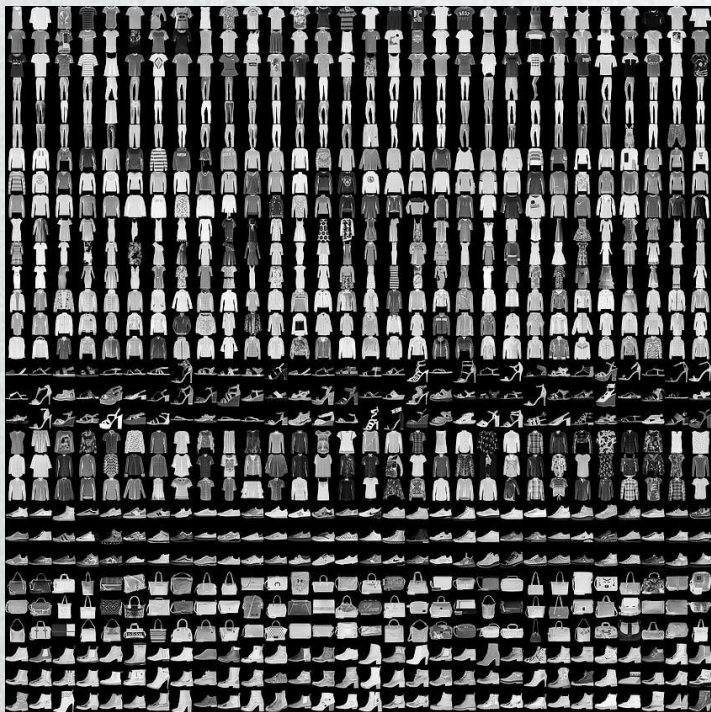
提纲



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. 简单卷积网络的搭建
- 3. 损失定义与模型优化
- 4. 图像识别大作业

商品图像 (Fashion-Mnist) 分类



FashionMNIST 是一个图像数据集。它是由 Zalando（一家德国的时尚科技公司）旗下的研究部门提供，涵盖了来自 10 种类别的共 7 万个不同商品的正面图片。

商品图像 (Fashion-Mnist) 分类



- 实现一个基于CNN的图像分类模型
- 具体要求：
 - 模型构建：继承nn.Module，实现FashionMnistModel类
 - 模型训练：基于Fashion-Mnist数据集，完成FashionMnistModel的训练
 - 模型测试：实现test函数，完成图像分类测试



机器学习的基本流程

数据准备

- 数据标注
- 训练集/验证集/测试集分割
- 特征提取

模型训练

- 分类损失函数
- 损失函数优化和参数调优

模型测试

- 性能评价指标
- 交叉验证

数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)
    """

    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
            if is_train:
                with gzip.open(paths[0], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
                with gzip.open(paths[1], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)
            else:
                with gzip.open(paths[2], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
                with gzip.open(paths[3], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)

        return img, label
```

数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

```
class FashionDataset(Dataset):
    '''
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)
    '''

    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
            if is_train:

                with gzip.open(paths[0], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
                with gzip.open(paths[1], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)
            else:
                with gzip.open(paths[2], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)

                with gzip.open(paths[3], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)

        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列

数据准备

- 构建数据集类：

构造函数：

实例化数据加载的相关类属性

获取所有数据个数

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)
    """

    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

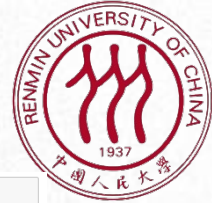
    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
            if is_train:
                with gzip.open(paths[0], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
                with gzip.open(paths[1], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)
            else:
                with gzip.open(paths[2], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
                with gzip.open(paths[3], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)

        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列



数据准备

• 构建数据集类：

构造函数：

实例化数据加载的相关类属性

获取所有数据个数

读取所有数据

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)
    """

    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
        if is_train:

            with gzip.open(paths[0], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
            with gzip.open(paths[1], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)
        else:
            with gzip.open(paths[2], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)

            with gzip.open(paths[3], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)

        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列

数据准备

- 实例化DataLoader:
 - Train_loader
 - Test_loader

```
# 定义data loader
train_dataset = FashionDataset('data',
                                transform=transforms.Compose([
                                    transforms.ToTensor(),
                                    transforms.Normalize((0.1307,), (0.3081,))
                                ])
)

train_loader = DataLoader(train_dataset, batch_size=320, shuffle=True, num_workers= 4)

test_dataset = FashionDataset('data',
                               transform=transforms.Compose([
                                   transforms.ToTensor(),
                                   transforms.Normalize((0.1307,), (0.3081,))
                               ]),
                               is_train = False
)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers= 1)
```



模型搭建与训练

- 模型训练与测试类：

```
class Model():
    def __init__(self):
        """
        创建模型和优化器，设置模型超参数
        * 参数
            * learning_rate
            * epoches
            * model_save_path
            * device: cuda or cpu
        * 模型
            * 创建FashionMnistModel的实例，命名为model
            * 定义optimizer
            * 定义loss function
        """
        self.lr = 0.01
        self.epoches = 20
        self.model_save_path = './model'
        # 指定训练的device, 优先使用GPU, GPU不可用时加载CPU
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = FashionMnistModel().to(self.device)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```



模型搭建与训练

- 模型训练与测试类：

```
class Model():
    def __init__(self):
        """
        创建模型和优化器，设置模型超参数
        * 参数
            * learning_rate
            * epoches
            * model_save_path
            * device: cuda or cpu
        * 模型
            * 创建FashionMnistModel的实例，命名为model
            * 定义optimizer
            * 定义loss function
        """
        self.lr = 0.01
        self.epoches = 20
        self.model_save_path = './model'
        # 指定训练的device, 优先使用GPU, GPU不可用时加载CPU
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = FashionMnistModel().to(self.device)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```

需同学自主完成

FashionMnistModel () 模型搭建



模型搭建与训练

- 模型搭建:

```
class FashionMnistModel(nn.Module):
    def __init__(self):
        '''
        *****请在此写入你的代码*****
        定义模型
        '''

    def forward(self, x):
        '''
        *****请在此处输入你的代码*****
        输入: input, 它的size是(batch_size, img_h, img_w, img_c)
        输出 (返回值): output(预测值), hidden(隐藏层的值)
            * output的size是(batch_size, num_label)

        定义模型函数:
        * 将输入经过卷积层和激活函数
        * 使用pooling降低通道数
        * 对卷积层的输出做适当的维度变换
        * 用线性层将output映射到num_label的维度上
        * 返回output
        '''
```




模型搭建与训练

- 模型训练:

```
def train(self, train_loader, test_loader):  
    '''  
    训练函数  
    '''  
    self.model.train()  
    for epoch in range(self.epochs):  
        loss_list = []  
        for batch_idx, (data, target) in enumerate(train_loader):  
            data, target = data.to(self.device), target.long().to(self.device)  
            self.optimizer.zero_grad()  
            output = self.model(data)  
            loss = self.loss_function(output, target)  
            loss.backward()  
            self.optimizer.step()  
            loss_list.append(loss.item())  
            if batch_idx % 50 == 0:  
                print('Train Epoch: {} [{}/{}] ({:.0f}%) \t Loss: {:.6f}'.format(  
                    epoch, batch_idx * len(data), len(train_loader.dataset),  
                    100. * batch_idx / len(train_loader), loss.item()))  
        self.test(test_loader)  
        # 保存模型参数  
        if epoch+1 % 5 == 0:  
            self._save_model(epoch+1)
```

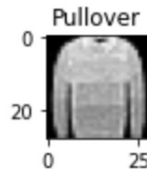
模型搭建与训练

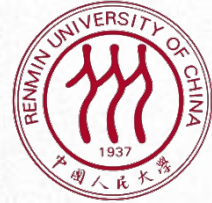
- 模型测试:

```
def test(self, test_loader):  
    '''  
    检验模型测试集上的效果  
    '''  
    self.model.eval()  
    test_loss = 0  
    correct = 0  
    with torch.no_grad():  
        for data, target in test_loader:  
            data, target = data.to(self.device), target.long().to(self.device)  
            output = self.model(data)  
            test_loss += self.loss_function(output, target).item() # sum up batch loss  
            pred = output.argmax(dim=1, keepdim=True) # get the index of the max log-probability  
            correct += pred.eq(target.view_as(pred)).sum().item()  
    test_loss /= len(test_loader.dataset)  
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%) \n'.format(  
        test_loss, correct, len(test_loader.dataset),  
        100. * correct / len(test_loader.dataset)))
```

```
img, label = test_dataset.__getitem__(20)  
pred = inference(model2, img)  
fig = plt.figure(figsize=(1, 1))  
plt.imshow(np.squeeze(img), cmap='gray')  
plt.title(label_classes[pred])
```

Text(0.5, 1.0, 'Pullover')



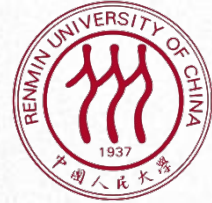


回顾



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. 简单卷积网络的搭建
- 3. 损失定义与模型优化
- 4. 图像识别大作业



谢谢！